

Construyendo de un silo a un Lakehouse: un problema de escalado

Cuándo vale la pena cambiar la arquitectura de datos

Diego Villalba

Almacenes y Minería de Datos

Facultad de Ciencias UNAM



**Facultad de
Ciencias**
UNAM

De silos a Lakehouse

Cuándo vale la pena cambiar la arquitectura de datos

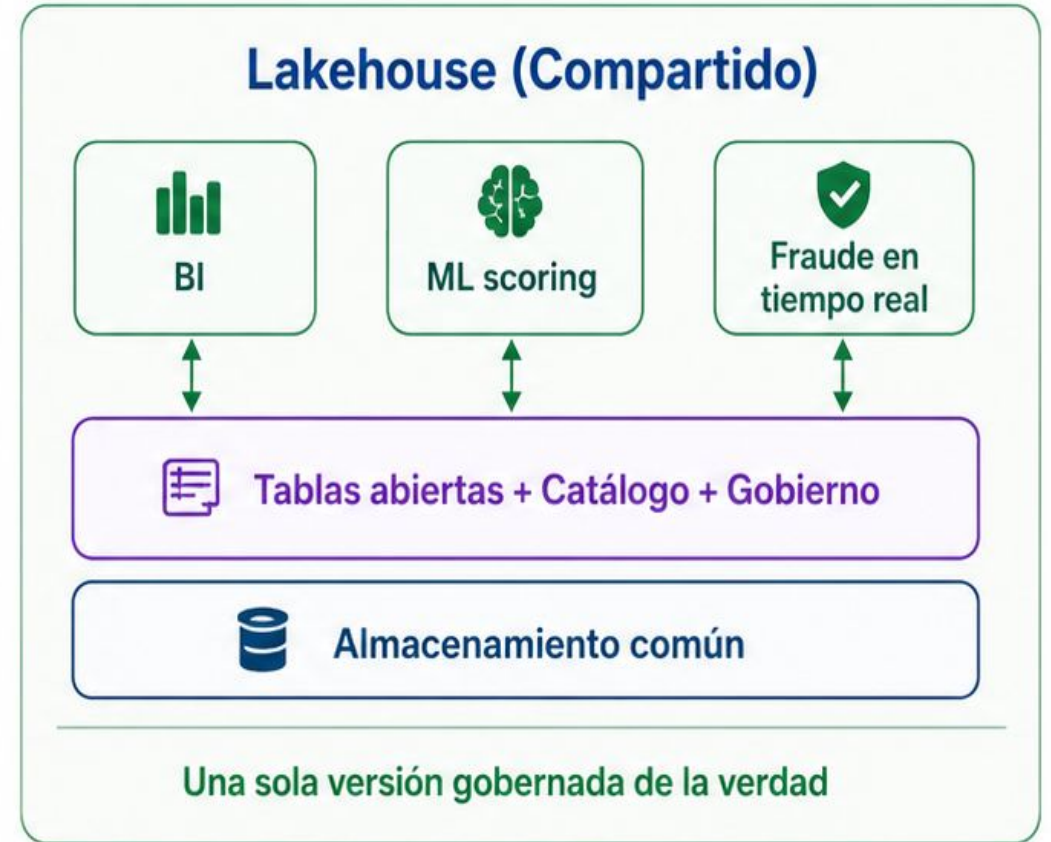
Caso de estudio: fintech de pagos móviles y préstamos digitales



➡

Migrar cuando
compartir datos
ya es crítico

Cambio de
arquitectura



La pregunta que guía la clase

¿Cuándo compartir una base abierta y gobernada para BI, streaming y ML justifica operar un Lakehouse?

Beneficio de compartir

- Menos copias
- Datos abiertos
- BI + ML + Streaming

Responsabilidad de operar

- Compactación
- Gobierno y permisos
- Contratos y observabilidad

Una transacción, dos respuestas

Transacción tx_8472 · \$3,200 · 10:02

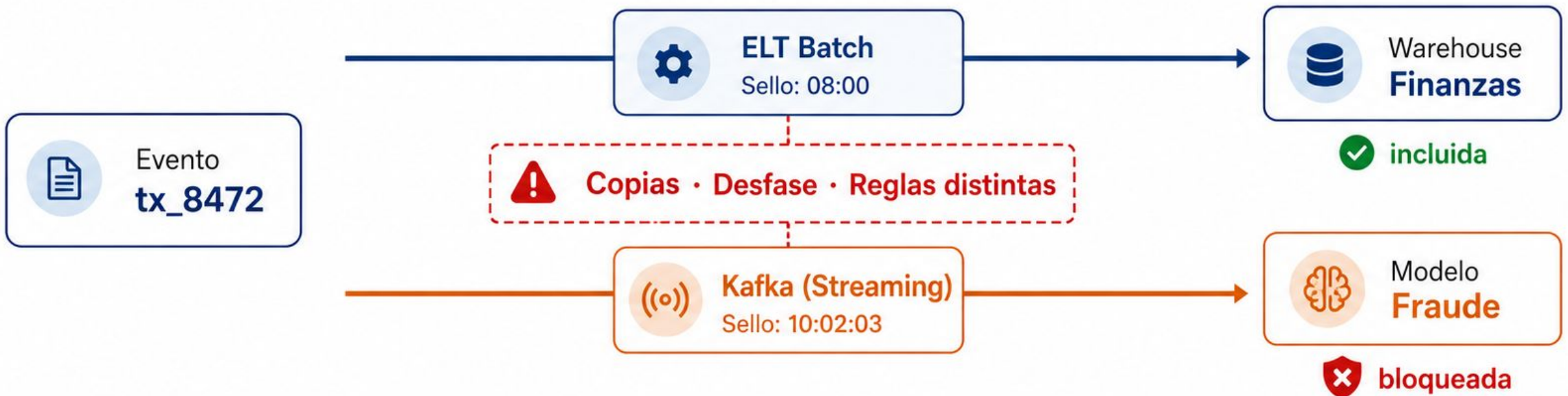


Finanzas: **incluida**



Fraude: **bloqueada**

¿Cómo pueden ser correctos ambos procesos y producir verdades distintas?



Idea clave

El problema no es que un proceso esté 'mal'; el problema es que trabajan sobre copias, tiempos y reglas distintas. Cuando esto afecta decisiones críticas, vale la pena migrar a una arquitectura compartida.



Facultad de Ciencias
UNAM

Dos sistemas especializados

Por qué Data Warehouse y Data Lake coexistieron durante años

Frontera organizacional
y técnica



Data Warehouse

- SQL y reportes
- Esquema controlado
- Alto desempeño analítico
- Menor flexibilidad

Optimizado para analítica estructurada y consumo de negocio.



Tablas



Almacenamiento
estructurado



Dashboards y
reportes



Data Lake

- Archivos y formatos diversos
- Esquema al leer
- Alta flexibilidad para exploración y ML
- Menor estandarización

Optimizado para datos crudos, exploración y ciencia de datos.



Archivos y
código



Documentos
(PDF)



Imágenes y
multimedia

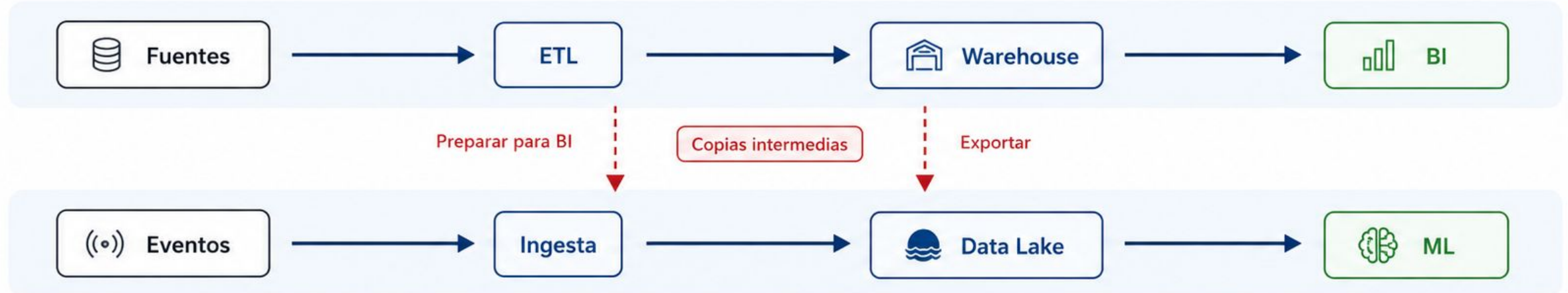


Idea clave

No eran rivales: cada uno resolvía una necesidad distinta dentro de la organización.

El impuesto de los silos

Cada copia de datos agrega costo, fricción y riesgo operativo



 Cada copia agrega:

-  Otro esquema
-  Otro control de calidad
-  Otra política de acceso
-  Sincronización
-  Más costo y mantenimiento

Más copias = más fricción operativa



Cuando este impuesto crece, vale la pena migrar a una arquitectura compartida.

Fintech, años 0-3

Stack típico cuando el negocio crece y aparecen necesidades analíticas



10,000 tx/día



≈500 GB/año

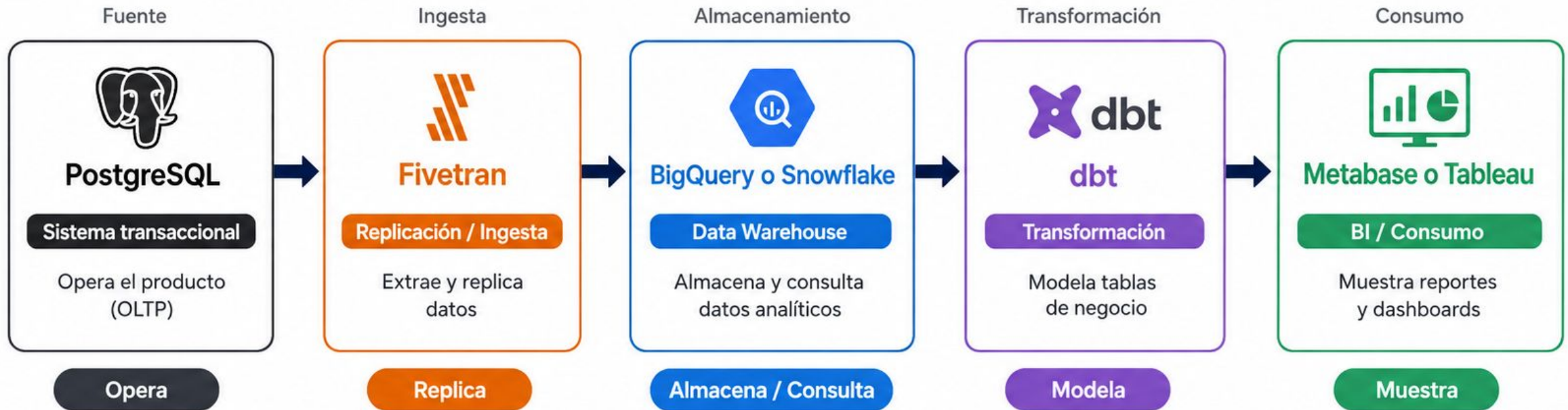


Datos relacionales



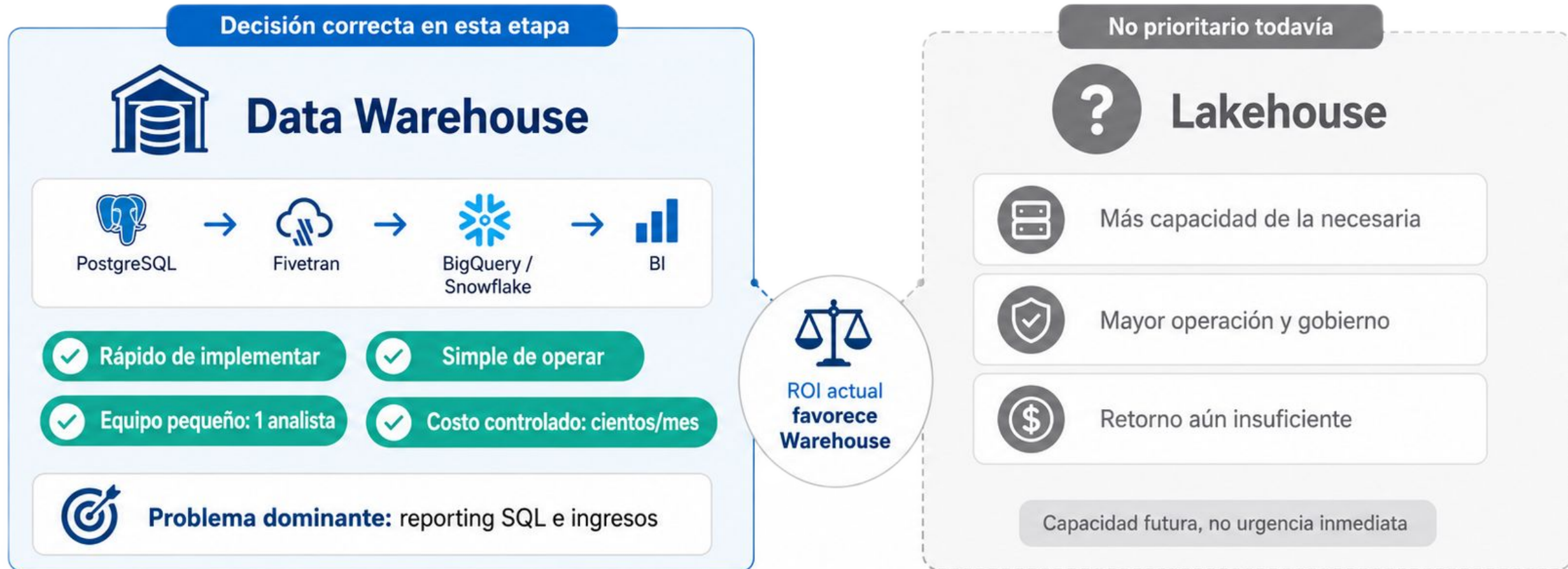
Reportes de ingresos

Sistemas que intervienen



Idea clave: en esta etapa no hay una arquitectura unificada; cada sistema resuelve una función específica del flujo analítico.

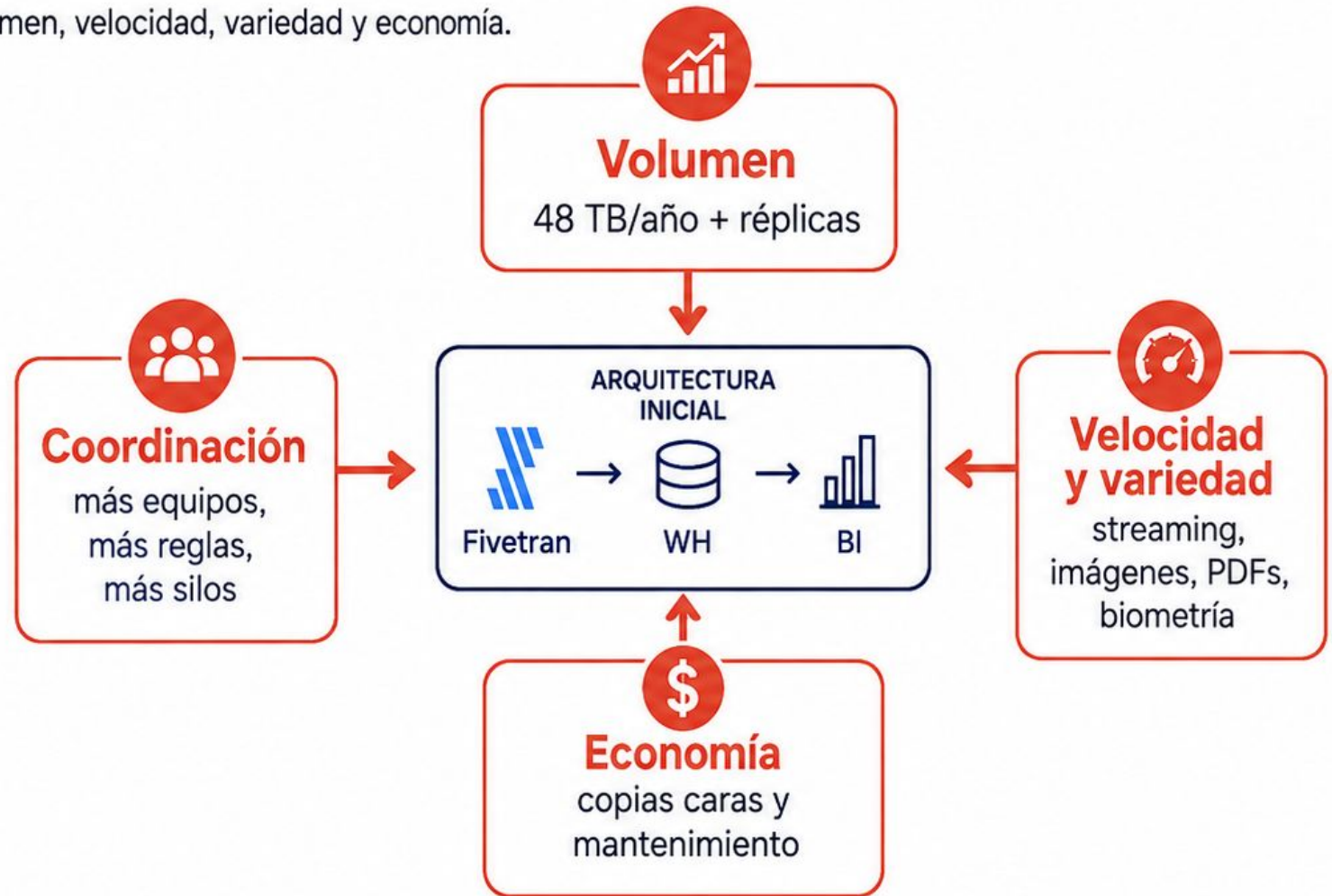
Por qué el Warehouse era correcto



Un Lakehouse todavía no ofrecía retorno suficiente para pagar su operación.

Año 4: cambió el problema

El stack inicial dejó de escalar cuando aparecieron volumen, velocidad, variedad y economía.



Ya no bastaba optimizar reportes: ahora el negocio necesitaba datos compartidos para BI, fraude y ML.

No elijamos una marca todavía



La elección depende del cuello de botella, no de la moda.



Objeto, bucket y key

Cómo interpretar una ruta y qué contiene realmente un objeto

`s3://` / `fintech-datos-prod` / `/documents/loan-8472.pdf`

↑ ↑ ↑

servicio bucket key del objeto



Conceptos clave



Bucket: contenedor con nombre para guardar objetos



Key: localizador único dentro del bucket



Objeto: bytes (contenido) + key + metadatos



Anatomía del objeto

Key: `/documents/loan-8472.pdf`

Bytes (Datos)



Metadata:

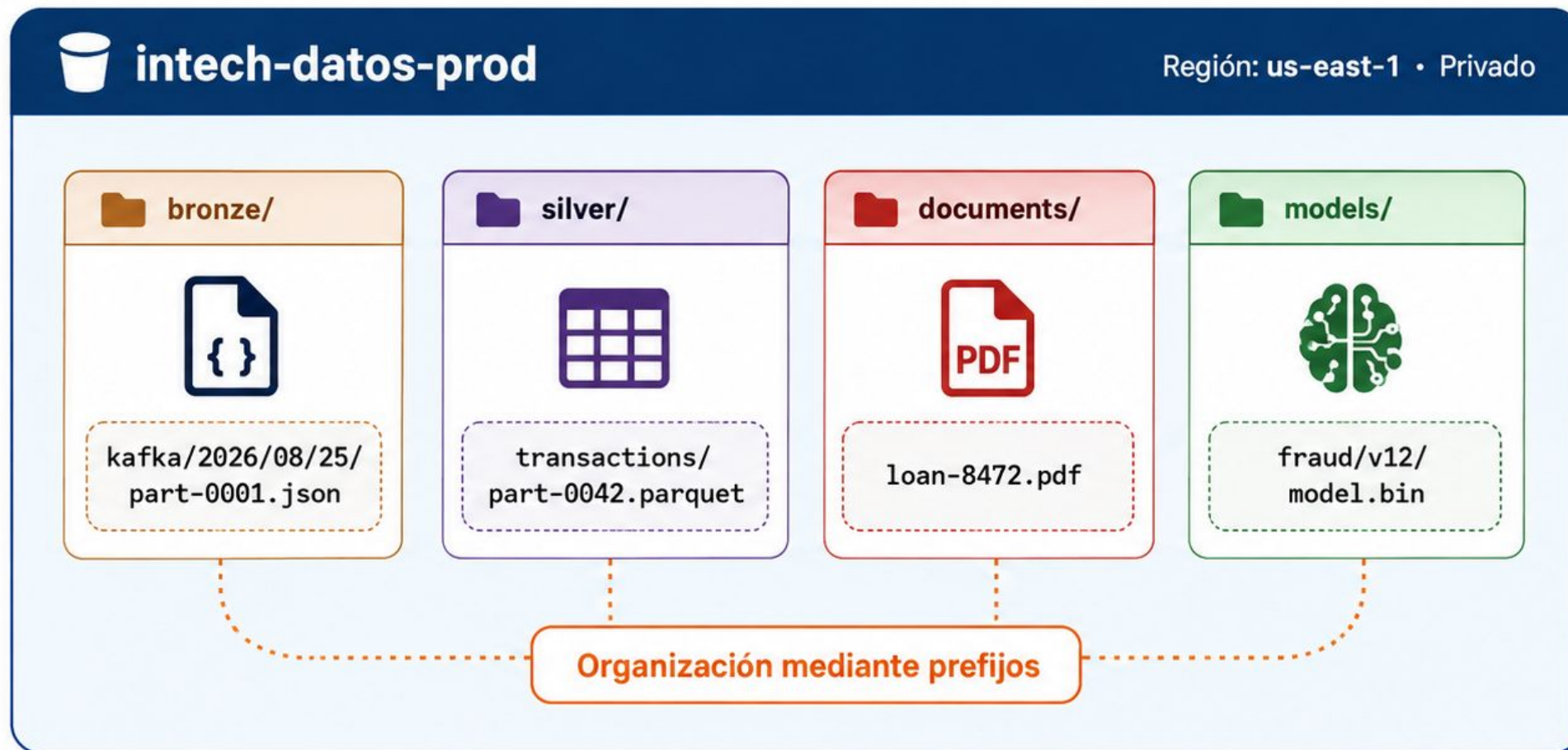
application/pdf · 680 KB · cifrado · PII



Un objeto no es solo un archivo: también incluye un identificador (key) y metadatos.

Anatomía de un bucket

Un bucket almacena objetos organizados por prefijos.



¿Qué significa?

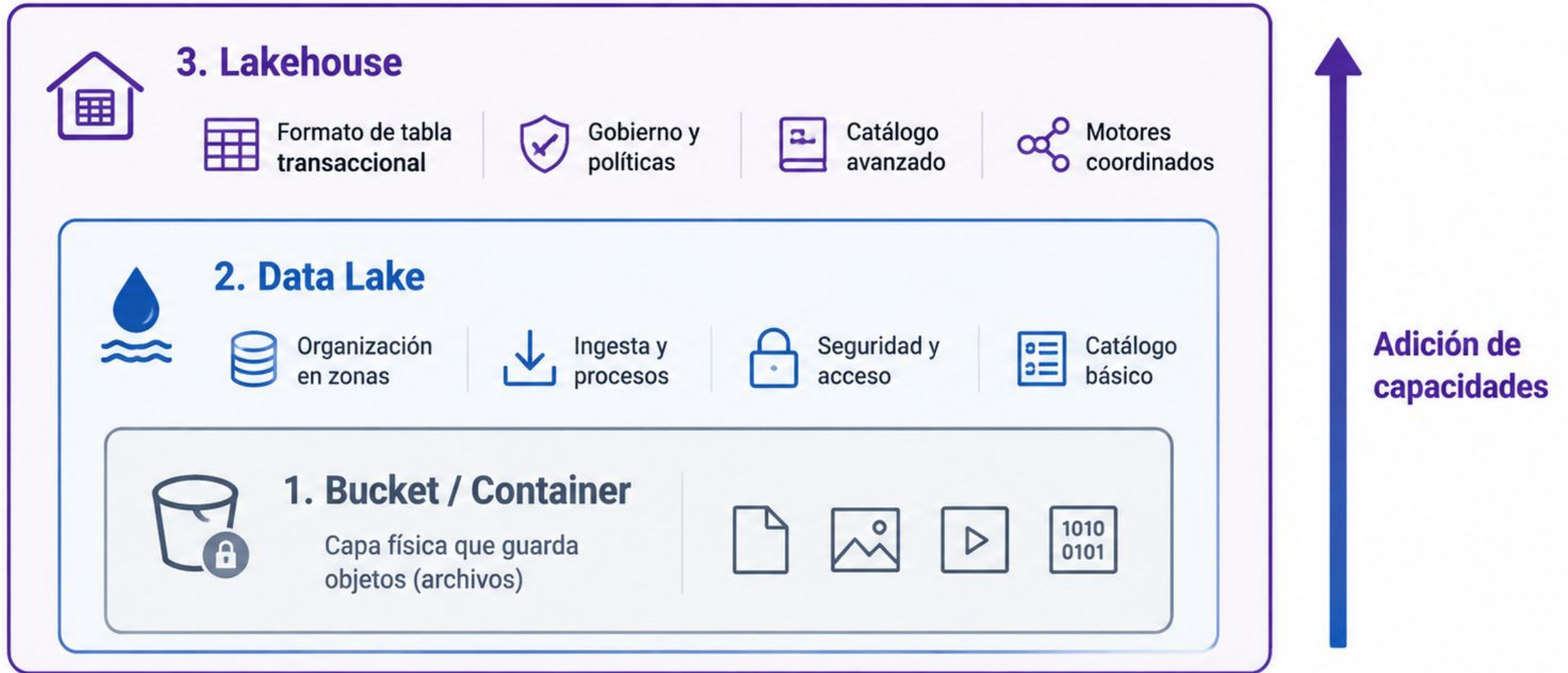
-  **Bucket** = contenedor
-  **Objeto** = archivo
-  **Prefijo** = convención de ruta

Los prefijos son convenciones de ruta para organizar objetos. No son carpetas reales ni tablas.



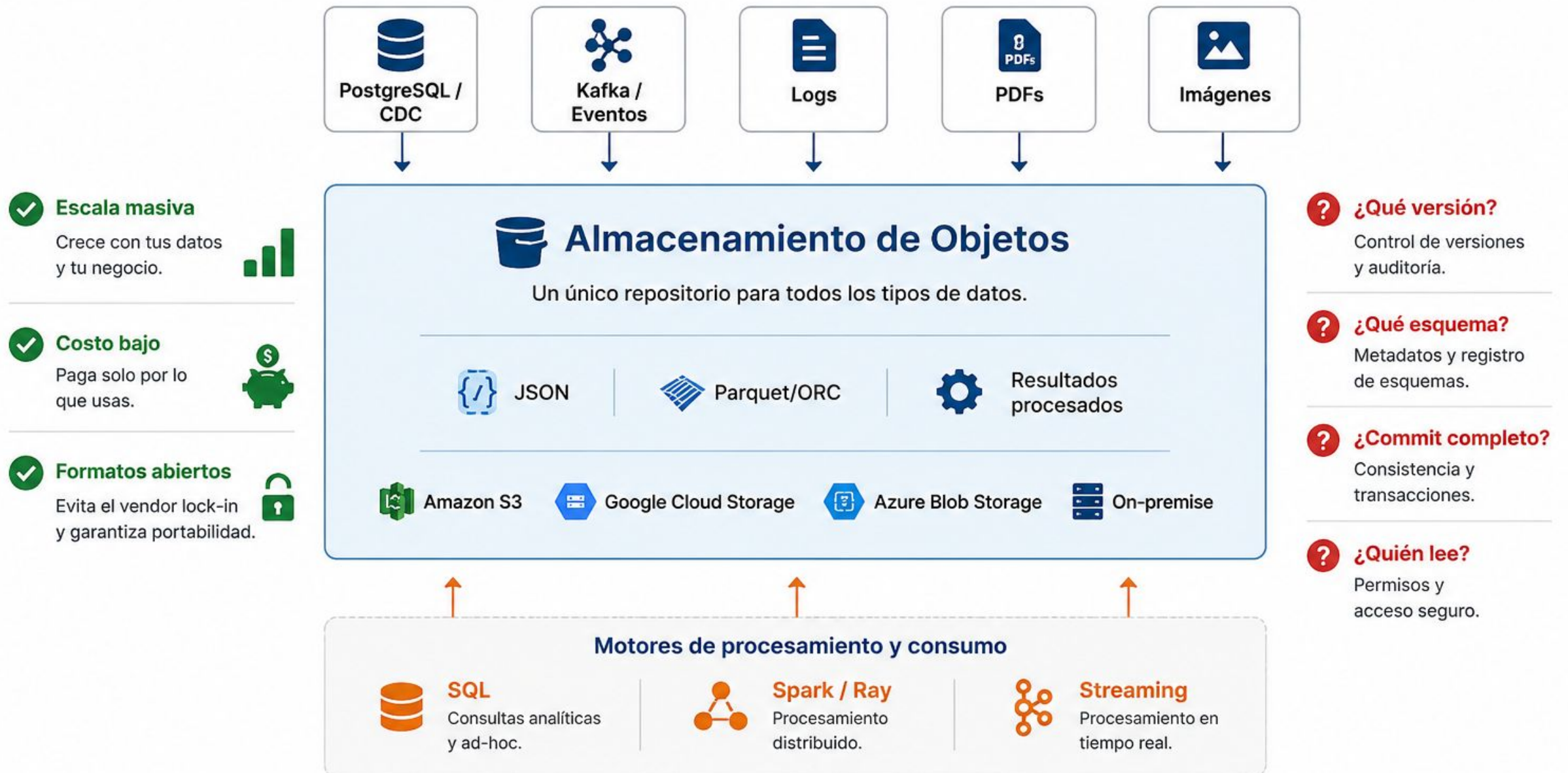
El bucket conserva objetos; no conoce por sí solo esquemas de tabla, JOINS ni commits.

Bucket, Data Lake y Lakehouse



Un Lakehouse no reemplaza al bucket: agrega organización, gobierno y tablas confiables sobre el Data Lake.

Una base física compartida



El bucket no sabe qué es una tabla

/transactions/				
part-0001.parquet		Archivo final	→	✓ Ok
part-0002.parquet		Archivo final	→	✓ Ok
part-0003.parquet		Archivo final	→	✓ Ok
part-0003-retry.parquet		Reintento / duplicado	→	⚠ ¿Duplicado?
_temporary-part-0004.parquet		Temporal / en escritura	→	⌚ ¿Temporal?

	¿Duplicado?	No hay control de versiones ni deduplicación	→	✗ No confiable
	¿Temporal?	No hay transacciones ni manejo de estados	→	✗ Riesgo de leer datos incompletos
	¿Escritura completa?	No hay atomicidad ni commits	→	✗ Posibles datos corruptos

Servicio faltante: MotorSQL

- Catálogo y metadatos**
Sabe qué archivos existen y su estado
- Transacciones ACID**
Garantiza atomicidad y consistencia
- Control de versiones (time travel)**
Histórico de cambios y rollback
- Optimización y deduplicación**
Elimina reintentos y archivos innecesarios

➔

Vista gestionada

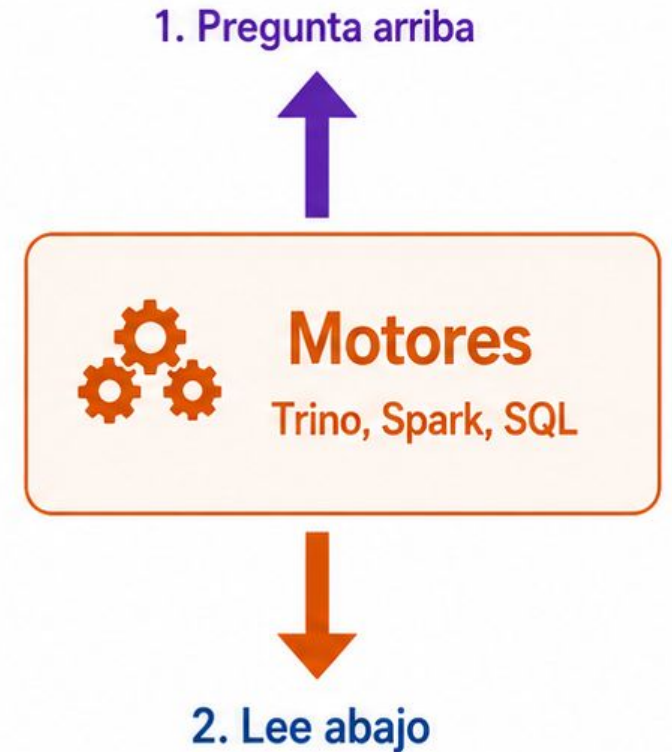
Archivo lógico	Estado	Versión	Tamaño	Commit
part-0001	COMMIT	v1	128 MB	✓
part-0002	COMMIT	v1	128 MB	✓
part-0003	COMMIT	v1	128 MB	✓
part-0003-retry	IGNORADO	-	-	✗
_temporary-part-0004	EN ESCRITURA	-	-	✗

✓

Resultado

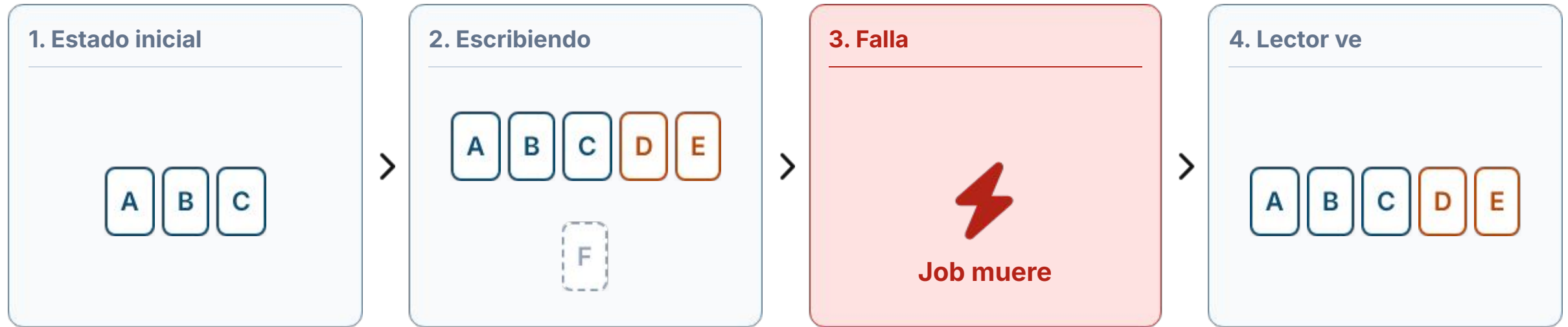
- Datos confiables
- Consultas consistentes
- Sin duplicados ni temporales
- Listo para BI y ML

Cuatro responsabilidades diferentes



El motor consulta arriba para entender la tabla y lee abajo para acceder a los datos.

Sin protocolo: escritura parcial



¿La actualización está completa? No puede saberlo. Mezcló datos válidos con datos a medias.

Con snapshots: publicar todo o nada

Los archivos nuevos se escriben primero y solo se publican cuando el commit termina con éxito.



Los lectores siguen viendo el snapshot publicado hasta que el job finaliza.

! Si falla

El puntero no cambia.
Los lectores siguen viendo Snapshot 17.



D E escritos pero invisibles

No se publica un estado parcial.

✓ Si tiene éxito (Commit)

El puntero se mueve atómicamente.

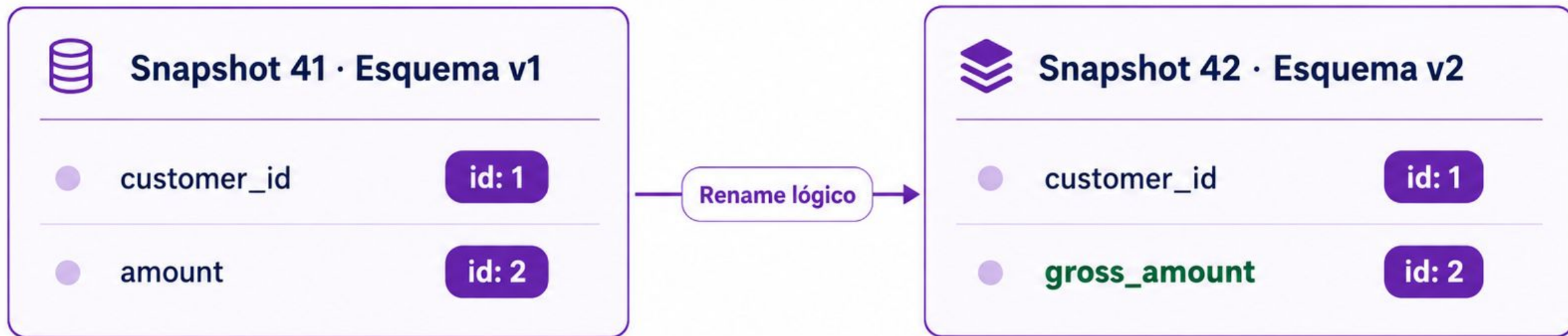


Se publica todo junto, de una sola vez.



Idea clave: un snapshot garantiza publicación atómica.
O los lectores ven la versión anterior completa, o ven la nueva completa.

Evolución de esquema



Al leer un archivo viejo, el motor usa el identificador de campo **id: 2** para encontrar los datos y les aplica el nombre vigente: **gross_amount**.

¡No fue necesario reescribir terabytes de archivos históricos!

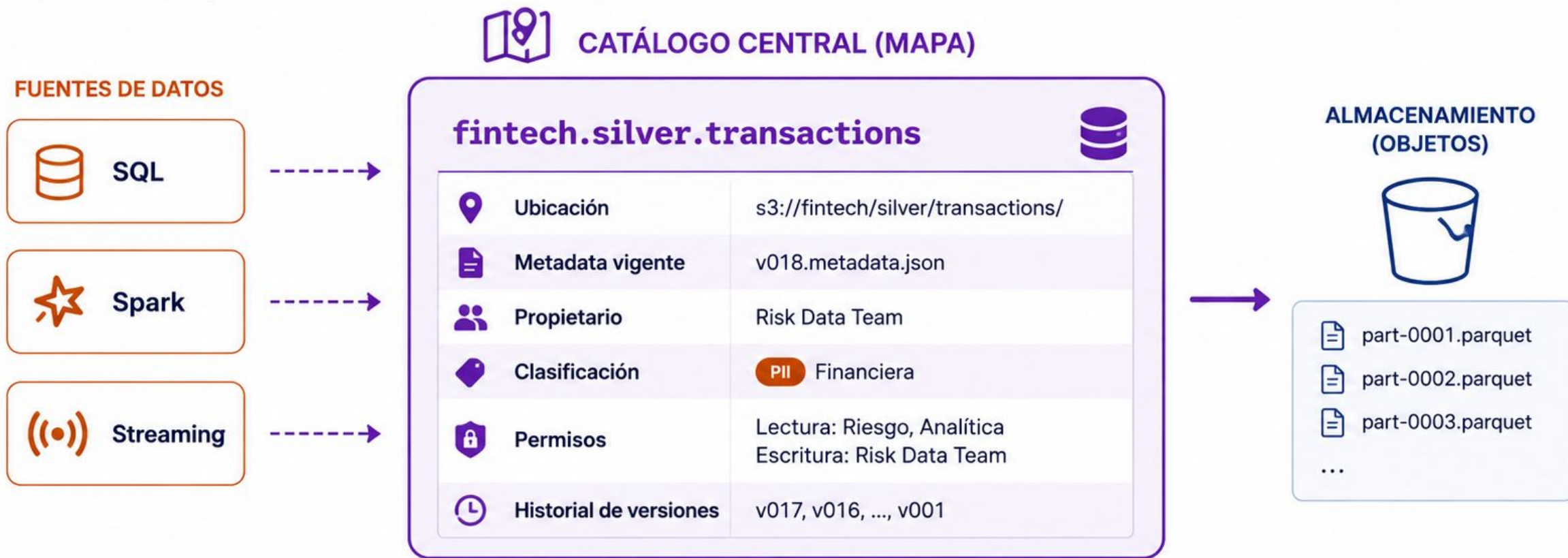


Los field IDs permiten compatibilidad entre versiones del esquema.

El soporte de estas operaciones depende del formato (Iceberg/Delta), la configuración y el motor utilizado.

El catálogo es el mapa central

El motor consulta el catálogo para resolver ubicación y permisos, y después lee los objetos directamente.



El catálogo es la fuente de verdad:



Encuentra rápidamente los datos correctos



Aplica permisos y gobierno



Mantiene metadata e historial de versiones



Permite motores y herramientas trabajar con confianza

Gobierno: quién puede ver qué

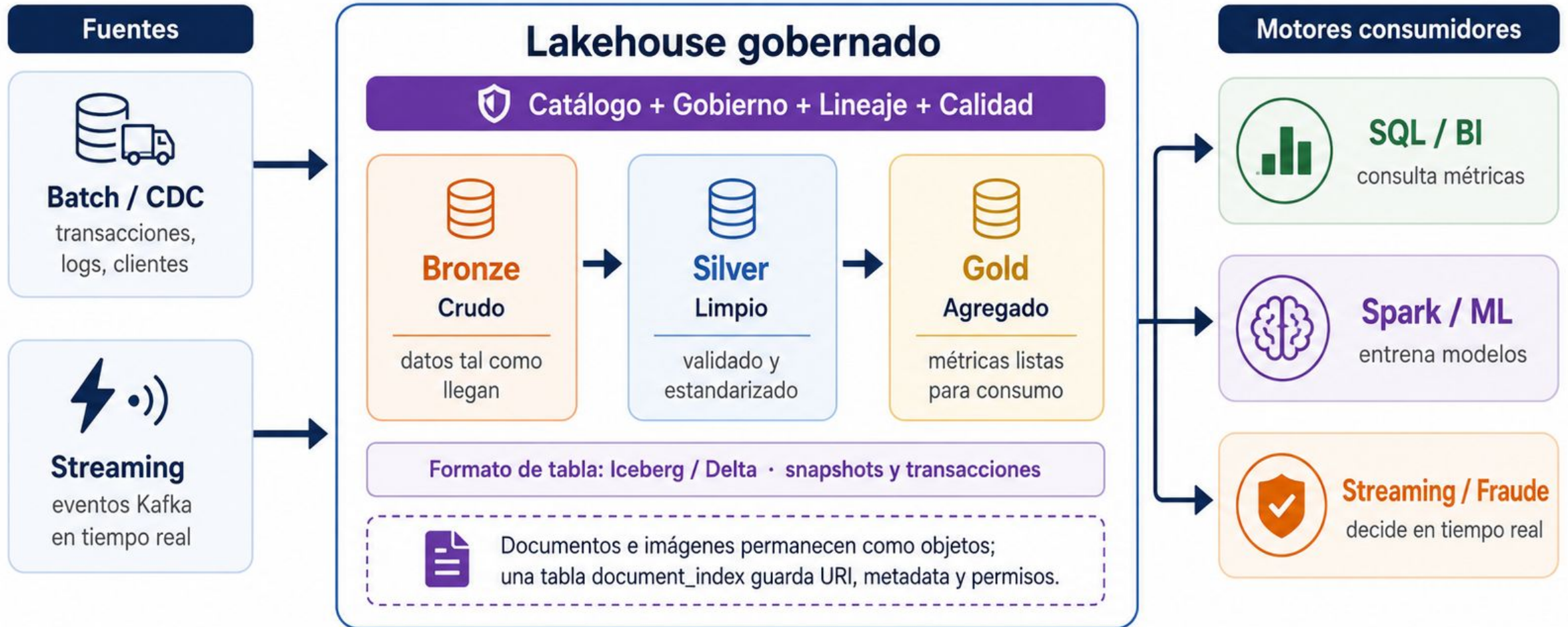
Control de identidad · Permisos · Linaje · Auditoría

Identidad	Gold Financiero	Silver Transacciones	Señales Dispositivo	Auditoría / Logs
Finanzas	✔ Lectura	🔒 Enmascarada	❌ Denegado	❌ Denegado
Fraude	❌ Denegado	✔ Lectura	✔ Lectura	❌ Denegado
Data Science	❌ Denegado	🏛 Seudonimizada	✔ Lectura	❌ Denegado
Auditoría	👁 Sólo lectura	👁 Sólo lectura	❌ Denegado	✔ Total

Ejemplos: Unity Catalog, AWS Glue + Lake Formation, Google BigLake. Conocer la ruta física (S3) no equivale a tener autorización.

Fintech, año 5: arquitectura Lakehouse

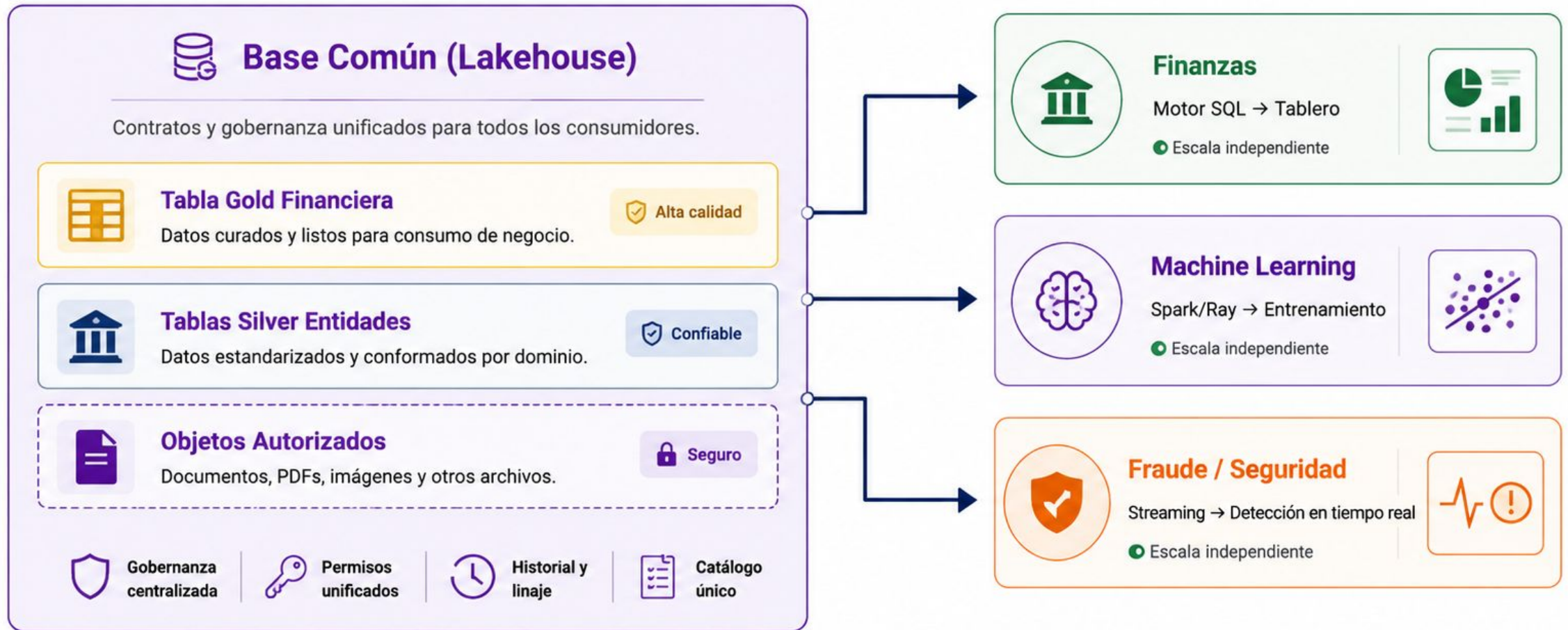
Los datos se organizan una vez y se consumen desde BI, ML y fraude sin crear nuevos silos.



Idea clave: no es una sola herramienta; es una base común gobernada que varios motores pueden usar.

Mismos contratos, distintos consumos

Una base común gobernada que cada motor consume según su necesidad y escala.



Beneficio clave: un solo contrato de datos, múltiples consumidores especializados.



Escala independiente por motor



Menor costo operativo



Sin duplicación de datos

Tecnologías por responsabilidad

Responsabilidad	Ejemplos (Leyenda: Formatos Abiertos Servicios Administrados)
Ingesta	Fivetran, Airbyte, Kafka, Kinesis, Pub/Sub
Warehouse (Silos)	BigQuery, Snowflake, Redshift, Fabric Warehouse
Objetos (Storage)	S3, Cloud Storage, ADLS Gen2, OneLake
Tabla (Formato)	Iceberg, Delta Lake, Hudi
Catálogo / Gobierno	Glue + Lake Formation, Unity Catalog, BigLake, Purview
Cómputo	Athena, Trino, BigQuery (sobre externo), Databricks SQL, Spark
Consumo	Tableau, Power BI, Looker, Metabase

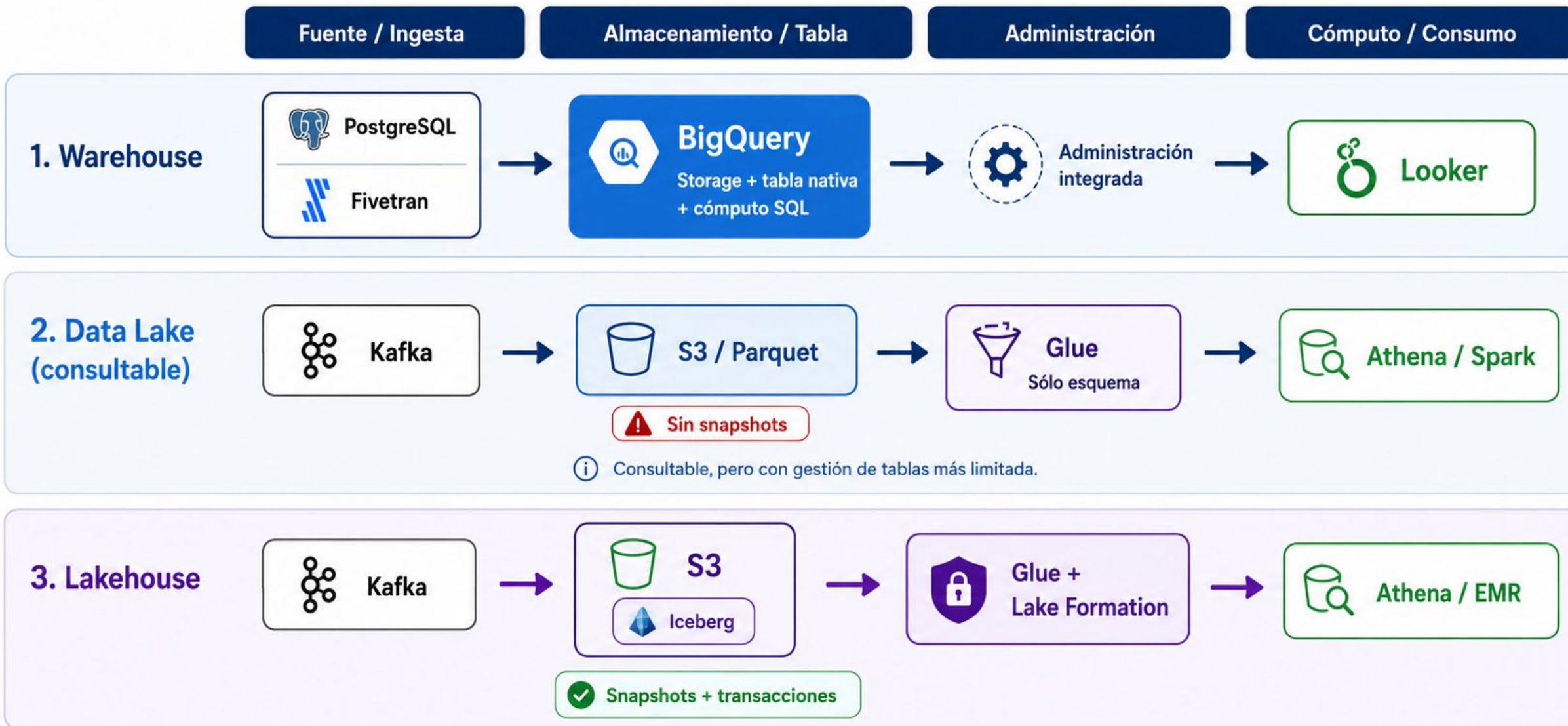
 S3 no es el Lakehouse completo.

 Iceberg/Delta/Hudi no son nubes.

 Un producto puede cubrir más de una función

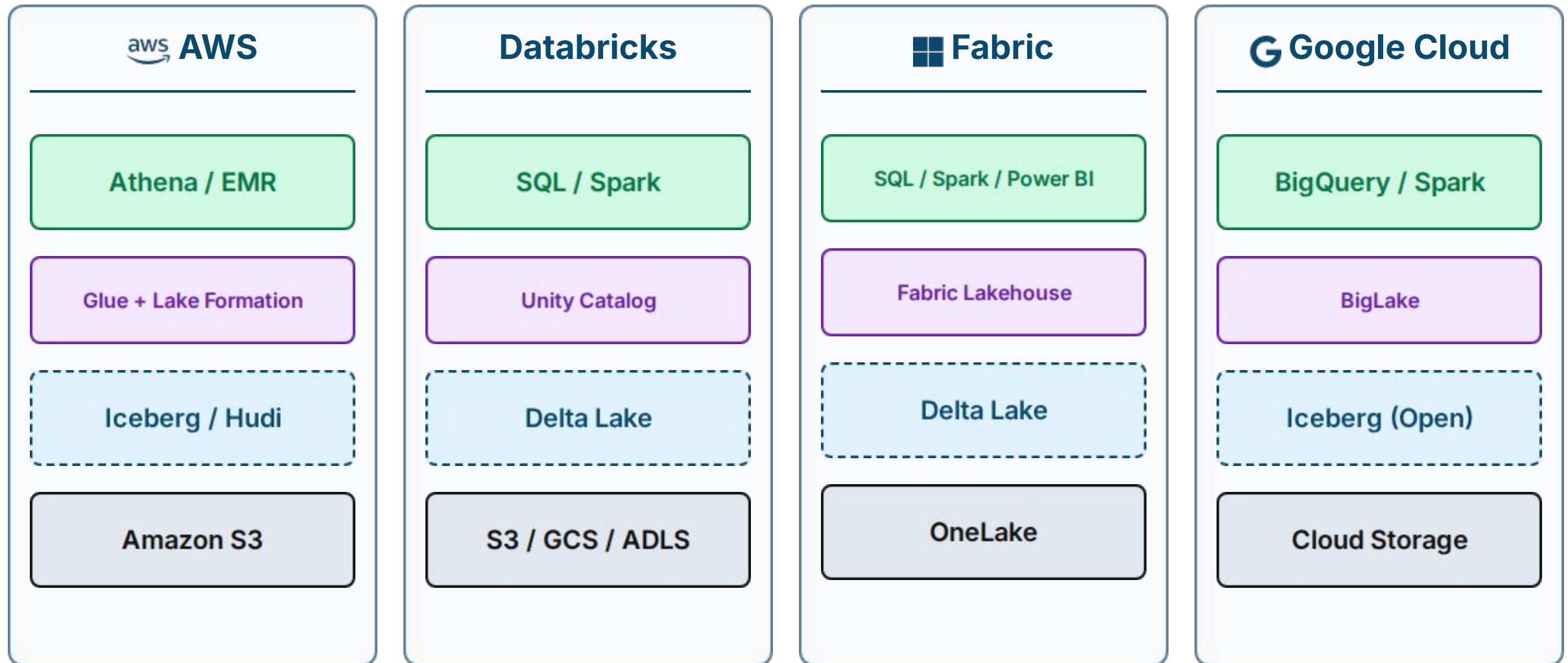
Tres patrones completos

Cómo se combinan fuente, almacenamiento, administración y consumo



Cuatro implementaciones de Lakehouse

Las marcas cambian; las responsabilidades permanecen.



Arquitectura por cuello de botella

Criterio	Warehouse	Lake + Warehouse	Lakehouse
Datos	Relacionales	Usos separados	Diversos y compartidos
Prioridad	SQL rápido	Aislar economías	Reducir copias
Operación	Menor carga	Sincronización doble	Gobierno y mantenimiento
Riesgo	Rigidez / costo	Dos verdades	Radio de impacto común

Años 0–3

Otra empresa puede decidir distinto

Año 5

Condición: Beneficio esperado > Carga operativa



Compartir datos también comparte el riesgo

En una base común, un cambio incorrecto puede impactar a varios consumidores al mismo tiempo.



Idea clave: una base compartida no elimina el riesgo; exige más gobierno para controlar el radio de impacto.



versionado y compatibilidad



validaciones antes de publicar



rollback y trazabilidad

La idea completa

Un Lakehouse gobernado, flexible y seguro.



Elección condicionada por el caso

No existe una arquitectura única para todos los casos de uso.



Menor costo total

Menos copias, más eficiencia en almacenamiento y cómputo.



Mayor agilidad

Nuevos casos de uso sin mover ni duplicar datos.



Escalabilidad

Cada motor escala de forma independiente.



Confianza

Datos correctos, auditables y listos para el negocio.



El Lakehouse no elimina la complejidad: la mueve y la gestiona.

Comprobación final

Completa las frases mentalmente:



1. Un bucket con Parquet no es todavía una tabla porque...



2. El catálogo sirve para..., mientras el formato de tabla sirve para...



3. La migración de la fintech se justifica cuando..., pero obliga a...

